COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	27/08/2026	**DATE SUBMITTED:**	28/08/2026
NAME:	Que, Josef Leroy P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Implement Ansible roles in playbooks**

1. PROCEDURE OUTPUT

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (*CPE232_yourname*). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issue this command.

```
queleroyubuntu3@workstation:~/CPE212_QUE$ git pull
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 777 bytes | 777.00 KiB/s, done.
From github.com:JosefLeroyQue/CPE212_QUE
   4a3e529..1752c44  main       -> origin/main
Updating 4a3e529..1752c44
Fast-forward
 ansible.cfg        | 11 ++++++++++
 install_apache.yml | 15 +++++++++++++++
 inventory          |  8 ++++++++
 3 files changed, 34 insertions(+)
 create mode 100644 ansible.cfg
 create mode 100644 install_apache.yml
 create mode 100644 inventory
queleroyubuntu3@workstation:~/CPE212_QUE$ ls
ansible.cfg  install_apache.yml  inventory  README.md
queleroyubuntu3@workstation:~/CPE212_QUE$
```

   Did something happen? Why? It pulled all of the files that are stored on my connected github repository.

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): *ansible-playbook --ask-become-pass install_apache.yml*. After

executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

```
PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
fatal: [centos]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to th
e host via ssh: leroyq@192.168.56.12: Permission denied (publickey,gssapi-keyex,gss
api-with-mic,password).", "unreachable": true}
ok: [node1]
ok: [node2]

TASK [update repository index] *************************************************
changed: [node1]
changed: [node2]

TASK [install apache2 package] *************************************************
changed: [node1]
changed: [node2]

TASK [add PHP support for apache] **********************************************
changed: [node1]
changed: [node2]

PLAY RECAP *********************************************************************
centos                     : ok=0    changed=0    unreachable=1    failed=0    skip
ped=0    rescued=0    ignored=0
node1                      : ok=4    changed=3    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
node2                      : ok=4    changed=3    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
```

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
    when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.
Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
fatal: [centos]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to th
e host via ssh: leroyq@192.168.56.12: Permission denied (publickey,gssapi-keyex,gss
api-with-mic,password).", "unreachable": true}
ok: [node2]
ok: [node1]

TASK [update repository index] *************************************************
changed: [node1]
changed: [node2]

TASK [install apache2 package] *************************************************
ok: [node1]
ok: [node2]

TASK [add PHP support for apache] **********************************************
ok: [node1]
ok: [node2]

PLAY RECAP *********************************************************************
centos                     : ok=0    changed=0    unreachable=1    failed=0    skip
ped=0    rescued=0    ignored=0
node1                      : ok=4    changed=1    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
node2                      : ok=4    changed=1    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
```

Analysis: It is the same result as the number as the centos does not support apt so it is still unreachable. What changed is the status for install apache2 package as well as add PHP support for apache.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

```
- name: update repository index
  apt:
    update_cache: yes
  when: ansible_distribution in [“Debian”, “Ubuntu]
```

Note: This will work also if you try. Notice the changes are highlighted.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
      stae: latest
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: update repository index
    dnf:
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install apache2 package
    dnf:
      name: httpd
      state: latest
    when: ansible_distribution == "CentOS"

  - name: add PHP support for apache
    dnf:
      name: php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
skipping: [centos]
ok: [node1]
ok: [node2]

TASK [add PHP support for apache] ***********************************************
skipping: [centos]
ok: [node1]
ok: [node2]

TASK [update repository index] **************************************************
skipping: [node1]
skipping: [node2]
ok: [centos]

TASK [install apache2 package] **************************************************
skipping: [node1]
skipping: [node2]
ok: [centos]

TASK [add PHP support for apache] ***********************************************
skipping: [node1]
skipping: [node2]
changed: [centos]

PLAY RECAP **********************************************************************
centos                     : ok=4    changed=1    unreachable=0    failed=0    skip
ped=3    rescued=0    ignored=0
node1                      : ok=4    changed=1    unreachable=0    failed=0    skip
ped=3    rescued=0    ignored=0
node2                      : ok=4    changed=1    unreachable=0    failed=0    skip
ped=3    rescued=0    ignored=0
```

Analysis: Several new tasks were added, their statuses showed they were skipping on them.

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

   5.1 To activate, go to the CentOS VM terminal and enter the following:
   systemctl status httpd
   The result of this command tells you that the service is inactive.

```
[leroyq@localhost ~]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d>
     Active: inactive (dead)
       Docs: man:httpd.service(8)

Aug 27 09:40:29 localhost.localdomain systemd[1]: Starting The Apache HTTP Serv>
Aug 27 09:40:29 localhost.localdomain httpd[93033]: AH00558: httpd: Could not r>
Aug 27 09:40:29 localhost.localdomain httpd[93033]: Server configured, listenin>
Aug 27 09:40:29 localhost.localdomain systemd[1]: Started The Apache HTTP Serve>
Aug 27 09:41:51 localhost.localdomain systemd[1]: Stopping The Apache HTTP Serv>
Aug 27 09:41:52 localhost.localdomain systemd[1]: httpd.service: Deactivated su>
Aug 27 09:41:52 localhost.localdomain systemd[1]: Stopped The Apache HTTP Serve>
lines 1-12/12 (END)
```

5.2 Issue the following command to start the service:

sudo systemctl start httpd

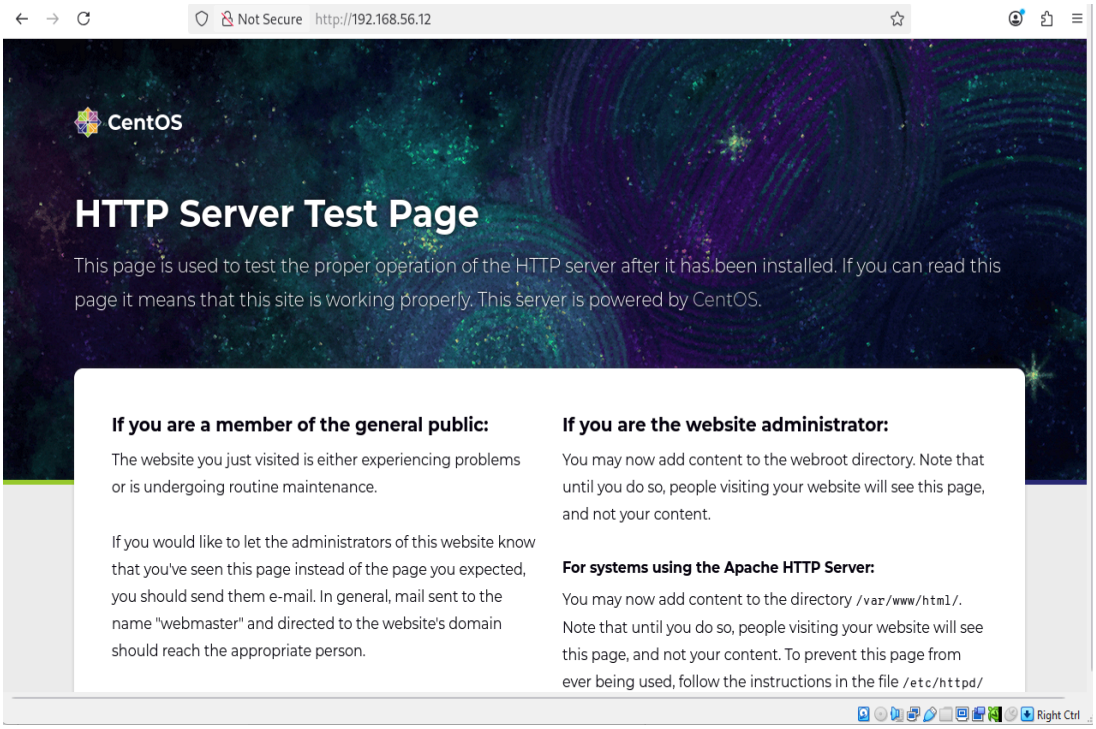
(When prompted, enter the sudo password)

sudo firewall-cmd --add-port=80/tcp

(The result should be a success)

```
[leroyq@localhost ~]$ sudo systemctl start httpd
[leroyq@localhost ~]$ sudo firewall-cmd --add-port=80/tcp
success
```

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index Ubuntu
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 and php packages for Ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: update repository index for CentOS
    dnf:
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install apache and php packages for CentOS
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
ok: [node1]
ok: [node2]
ok: [centos]

TASK [update repository index Ubuntu] ******************************************
skipping: [centos]
changed: [node2]
changed: [node1]

TASK [install apache2 package and php packages for Ubuntu] *********************
skipping: [centos]
ok: [node1]
ok: [node2]

TASK [update repository index for CentOS] **************************************
skipping: [node1]
skipping: [node2]
ok: [centos]

TASK [install apache and php packages for CentOS] ******************************
skipping: [node1]
skipping: [node2]
ok: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=3    changed=0    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
node1                      : ok=3    changed=1    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
node2                      : ok=3    changed=1    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
```

Analysis: Since we now grouped the required package names into YAML lists under the single task definitions, the playbook structure was simplified into 4.

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:

```
---
- hosts: all
  become: true
  tasks:

   - name: install apache2 and php packages for Ubuntu
     apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
     when: ansible_distribution == "Ubuntu"

   - name: install apache and php packages for CentOS
     dnf:
       name:
         - httpd
         - php
       state: latest
       update_cache: yes
     when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
ok: [node1]
ok: [node2]
ok: [centos]

TASK [update repository index Ubuntu] ******************************************
skipping: [centos]
changed: [node2]
changed: [node1]

TASK [install apache2 package and php packages for Ubuntu] *********************
skipping: [centos]
ok: [node1]
ok: [node2]

TASK [update repository index for CentOS] **************************************
skipping: [node1]
skipping: [node2]
ok: [centos]

TASK [install apache and php packages for CentOS] ******************************
skipping: [node1]
skipping: [node2]
ok: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=3    changed=0    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
node1                      : ok=3    changed=1    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
node2                      : ok=3    changed=1    unreachable=0    failed=0    skip
ped=2    rescued=0    ignored=0
```

Analysis: Here we can see that the playbook successfully executed all throughout the nodes without any errors. For the execution, we can see the when condition based on how it skipped the CentOS-specific repository and package tasks on node 1 and 2, while also skipping the Ubuntu-specific repository and package tasks on the centos node.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the apache_package and php_package are variables. The names are arbitrary, which means we can choose different names. We also take out the line when: ansible_distribution. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit.

```
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php
    apt:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
 GNU nano 6.2                          install_apache.yml
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php
    apt:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

```
queleroyubuntu3@workstation:~/CPE212_QUE$ ansible-playbook --ask-become-pass instal
l_apache.yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [node2]
ok: [node1]
ok: [centos]

TASK [install apache and php] **************************************************
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [centos]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Er
rno 2] No such file or directory: b'apt-get'", "rc": 2}
ok: [node2]
ok: [node1]

PLAY RECAP *********************************************************************
centos                     : ok=1    changed=0    unreachable=0    failed=1    skip
ped=0    rescued=0    ignored=0
node1                      : ok=2    changed=0    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
node2                      : ok=2    changed=0    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
```

Analysis: Here, the centos node should fail since it is an RPM-based distribution which means it is dependent on dnf or yum and it lacks the apt-get binary which leads to not executing the module on CentOS.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
192.168.56.120 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.121 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.122 apache_package=httpd php_package=php
```

Make sure to save the *inventory* file and exit.

```
  GNU nano 6.2                          inventory.ini
[managed_nodes]
node1 ansible_host=192.168.56.9 ansible_user=queleroyubuntu3 apache_package=apache>
node2 ansible_host=192.168.56.10 ansible_user=queleroyubuntu3 apache_package=apach>
centos ansible_host=192.168.56.12 ansible_user=leroyq apache_package=httpd php_pac>
```

Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assign as apt, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host

or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: ansible.builtin.package – Generic OS package manager — Ansible Documentation

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
queleroyubuntu3@workstation:~/CPE212_QUE$ ansible-playbook --ask-become-pass instal
l_apache.yml
BECOME password:

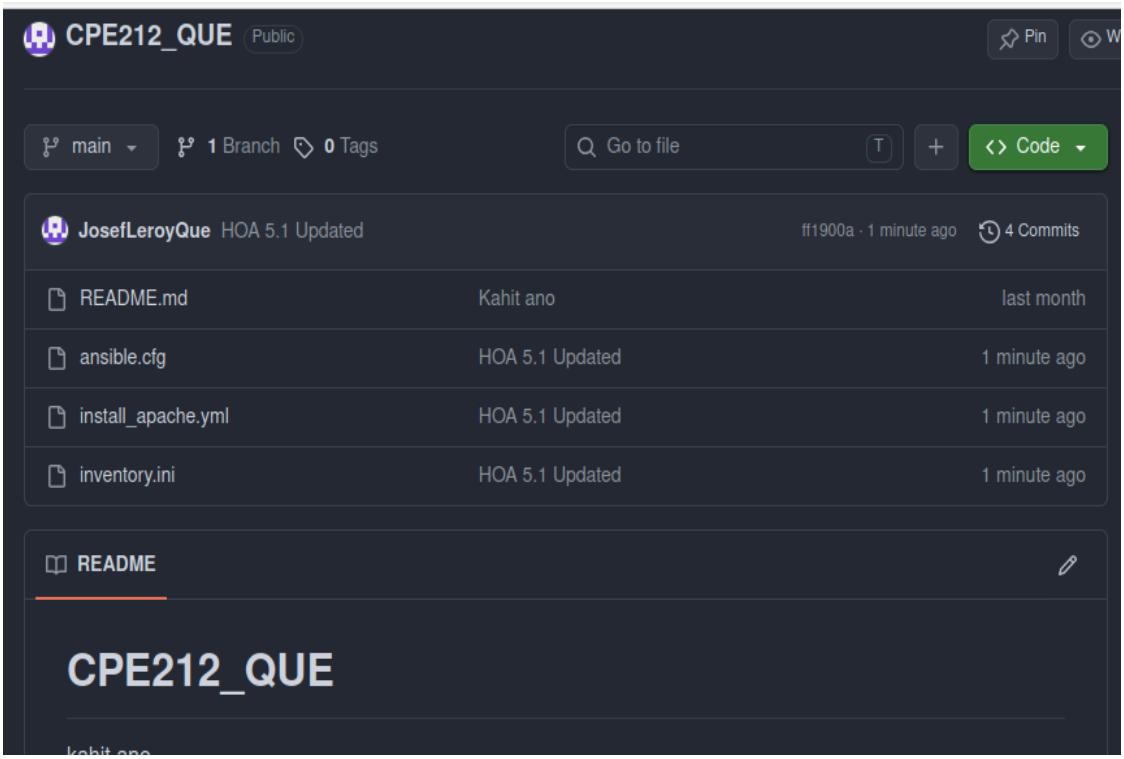
PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [node1]
ok: [node2]
ok: [centos]

TASK [install apache and php] **************************************************
ok: [node1]
ok: [node2]
ok: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=2    changed=0    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
node1                      : ok=2    changed=0    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
node2                      : ok=2    changed=0    unreachable=0    failed=0    skip
ped=0    rescued=0    ignored=0
```

Git commit:


CPE212_QUE
Public
Pin
main
1 Branch
0 Tags
Go to file
T
Code
JosefLeroyQue HOA 5.1 Updated
ff1900a · 1 minute ago
4 Commits
README.md
Kahit ano
last month
ansible.cfg
HOA 5.1 Updated
1 minute ago
install_apache.yml
HOA 5.1 Updated
1 minute ago
inventory.ini
HOA 5.1 Updated
1 minute ago
README
CPE212_QUE

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

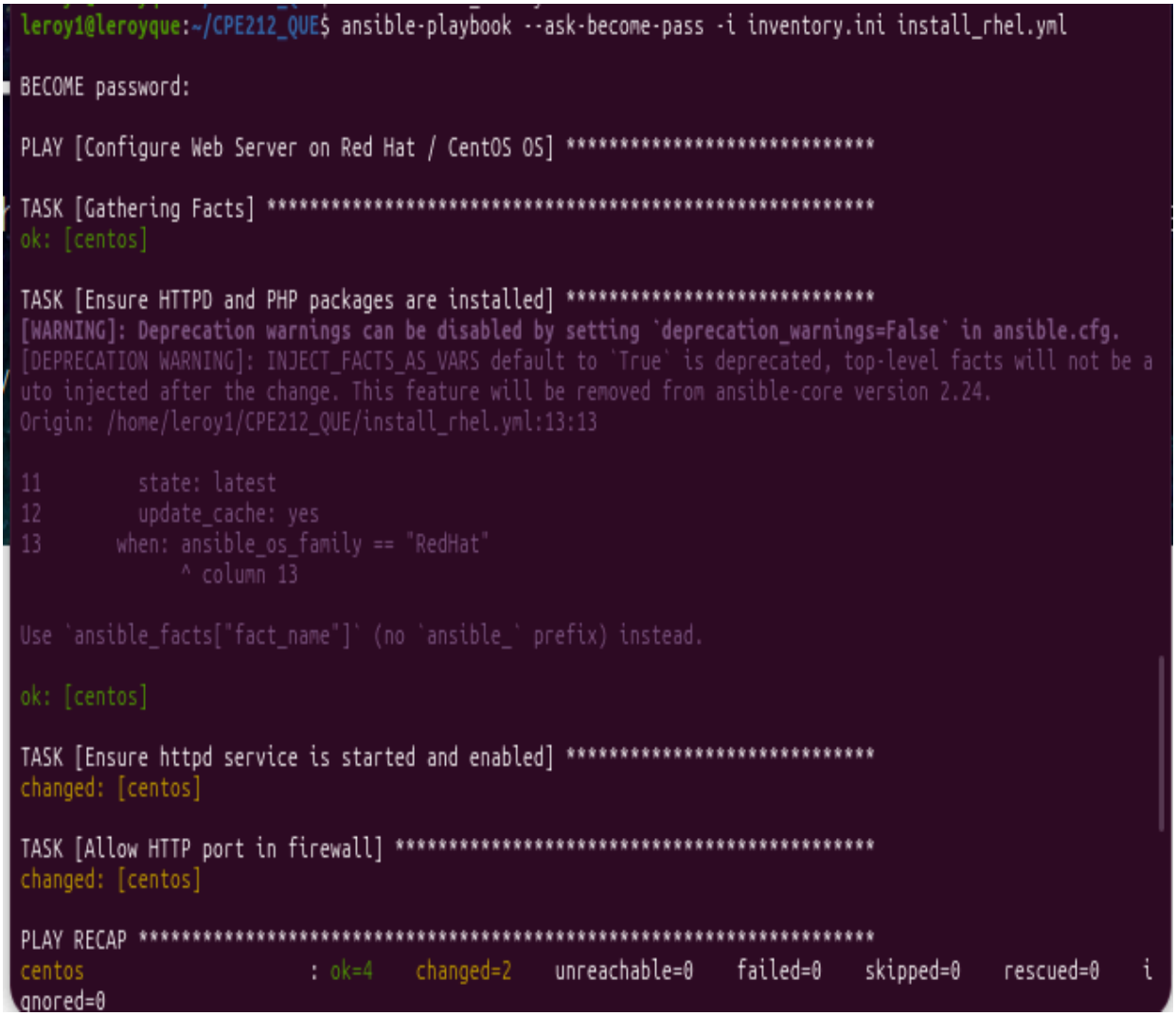
Supplementary Activity:

1. Create a playbook that could do the previous tasks in Red Hat OS.

```
leroy1@leroyque:~/CPE212_QUE$ cat install_rhel.yml
---
- name: Configure Web Server on Red Hat / CentOS OS
  hosts: centos
  become: true
  tasks:
    - name: Ensure HTTPD and PHP packages are installed
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
      when: ansible_os_family == "RedHat"

    - name: Ensure httpd service is started and enabled
      service:
        name: httpd
        state: started
        enabled: yes

    - name: Allow HTTP port in firewall
      ansible.posix.firewalld:
        port: 80/tcp
        permanent: yes
        state: enabled
        immediate: yes
      ignore_errors: yes
```



```
leroy1@leroyque:~/CPE212_QUE$ ansible-playbook --ask-become-pass -i inventory.ini install_rhel.yml

BECOME password:

PLAY [Configure Web Server on Red Hat / CentOS OS] ****************************

TASK [Gathering Facts] ********************************************************
ok: [centos]

TASK [Ensure HTTPD and PHP packages are installed] ****************************
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated, top-level facts will not be a
uto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /home/leroy1/CPE212_QUE/install_rhel.yml:13:13

11         state: latest
12         update_cache: yes
13       when: ansible_os_family == "RedHat"
               ^ column 13

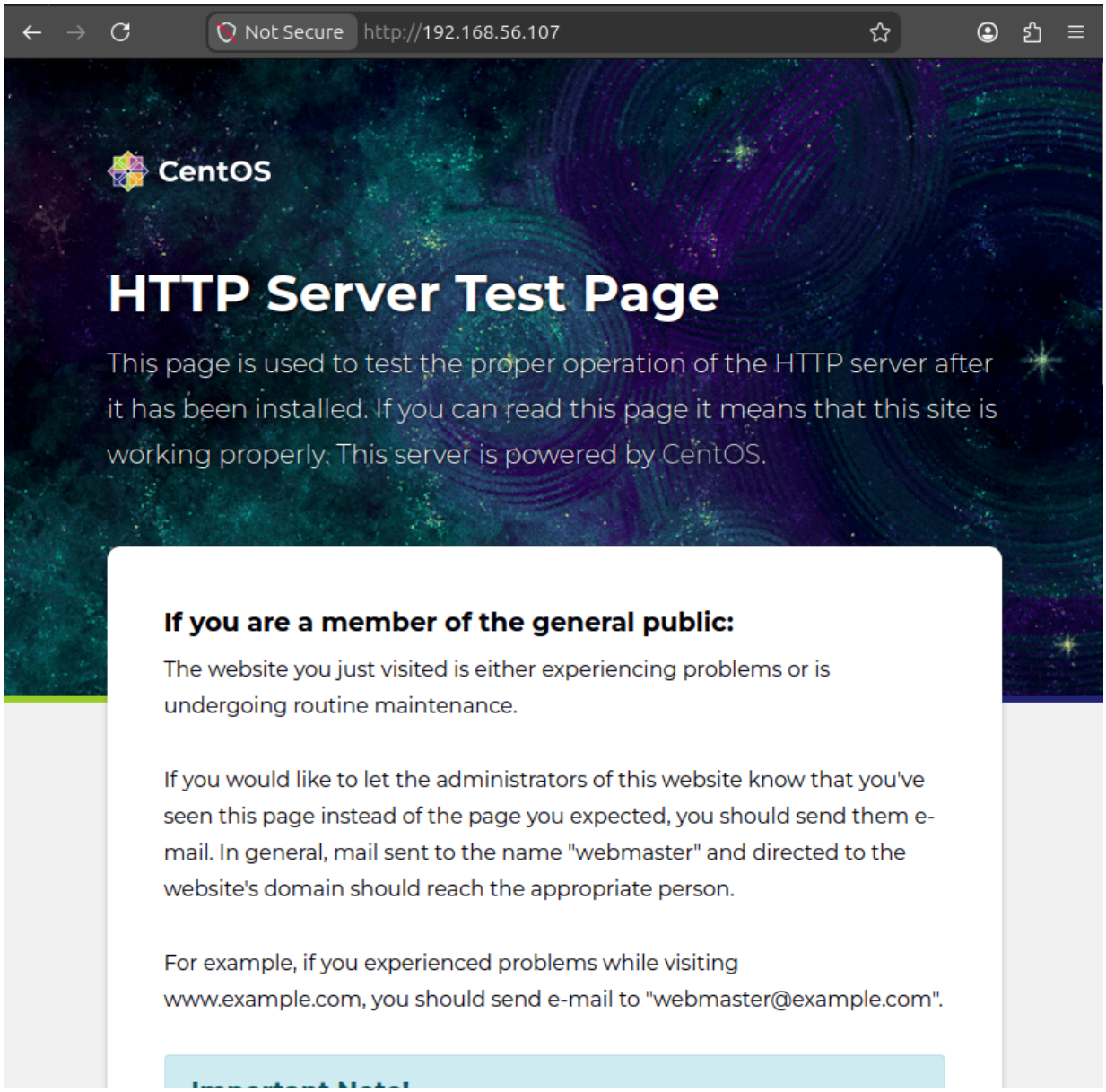
Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.

ok: [centos]

TASK [Ensure httpd service is started and enabled] ****************************
changed: [centos]

TASK [Allow HTTP port in firewall] ********************************************
changed: [centos]

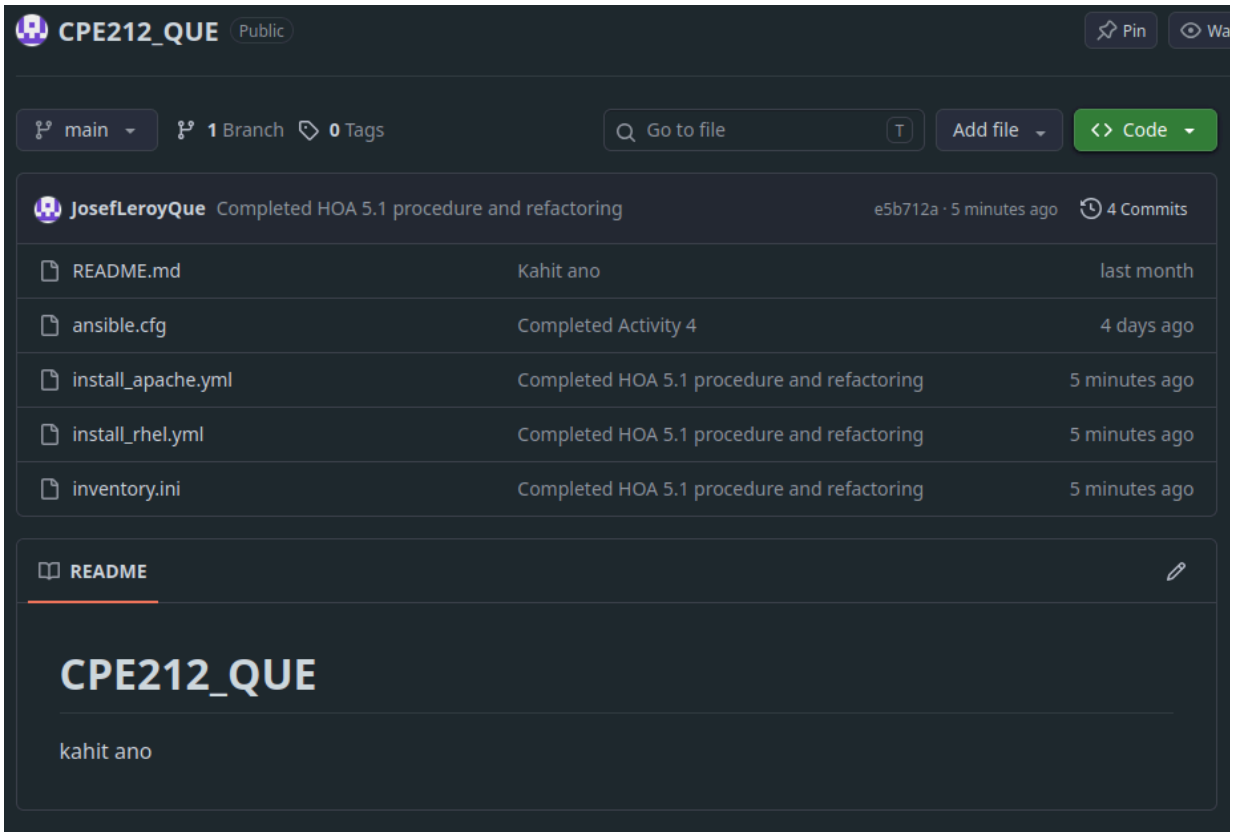
PLAY RECAP ********************************************************************
centos                     : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    i
gnored=0
```



Analysis: Please be advised that the supplementary part is done on my personal PC which changes my IP addresses and usernames for my VMs. Now for the supplementary part task, I constructed an OS-based playbook entitled install_rhel.yml with clear intentions of having it execute configuration on my CentOS virtual machine. This playbook was set up in such a way that the dnf module was utilized to install httpd and php while the operation of the Apache service was guaranteed through the service module with the help of which a task was added in order to open port 80 via firewalld.

When the playbook was run, Ansible had finished its job reporting on zero failures (failed=0) and listing too many changes (changed=2), confirming the successful startup of Apache as well as the application of firewall rules. To verify how everything was functioning, I went to http://192.168.56.107 through my browser and what I saw there was the default CentOS HTTP Server Test Page, which proved that the playbook did not only install the software but also effectively managed network rules and system services in order for the web server to get launched.

Git commit:



Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?
   - Refactoring playbook codes is crucial as it converts code that consists of hard-coded and repetitive tasks into one that is scalable, manageable, and highly effective. Through refactoring, distribution-specific modules are replaced by generic modules like package, and tasks are combined into tidy YAML lists. This means that one playbook can now control different environments that are based on Debian as well as Red Hat operations without leading to errors. Furthermore, making updates to the repository directly results in reduced overhead involved in executing the operations and increases the ease of reading and troubleshooting the code.
2. When do we use the “when” command in playbook?
   - A "when" command is used in Ansible whenever a given task has to be executed conditionally depending on specific facts about the system or state variables that are collected at the runtime. It is frequently found in multi-OS infrastructures as it is used to run tasks that are related to a specific platform, such as executing dnf modules only in relation to Red Hat unit and skipping it on Ubuntu. In addition to applying it in the contexts of the operational system, a when statement is also useful when the company is utilizing deployments across different environments.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was solely used for explanation of concepts for me especially when an unfamiliar word were introduced.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.